

Email Spam Detector

Student Learning Version

The Goal

The objective of this project is to build a machine learning model that can read an email and classify it as "Spam" or "Ham" (Not Spam). We also want to wrap this model in a web interface so normal users can interact with it.

Step 1: Preparing the Data (TF-IDF)

Computers can't read words like humans do; they only understand numbers. So, we have to convert the text into numbers using **TF-IDF**.

- **TF (Term Frequency)**: How often does a word appear in *this* specific email?
 - **IDF (Inverse Document Frequency)**: How often does this word appear across *all* emails in the dataset? If a word like "the" appears everywhere, it's not useful. If a word like "Winner" appears only in a few emails, it's very useful!
- TF-IDF multiplies these together to score every word.

Step 2: The Model (Random Forest)

We use a **Random Forest Classifier**. Imagine a single "Decision Tree" as a flowchart. It asks: *Does the email contain "Money"? If yes, is it written in all capital letters?* A single tree might make mistakes, so a Random Forest builds *hundreds* of these trees and makes them vote. If 90 out of 100 trees vote "Spam", the model predicts "Spam" with 90% confidence! Our model achieves over **97.31% accuracy**.

Step 3: Explainability (XAI)

Nobody likes a "black box" AI. We want to know *why* the model made its choice. Scikit-Learn provides `feature_importances_`. We map these importance scores back to the words they represent. This allows the web app to show the user exactly which words triggered the spam filter!

Step 4: The Web App & Premium UI

We built a web server using **Flask**. It creates an endpoint `/check` where the frontend sends the email text. Flask hands the text to the Vectorizer, then the Model, and sends the final prediction back to the user's browser. The frontend itself was engineered using modern web-design principles, featuring a deep space dark mode and dynamic gradient styling.

Step 5: Continuous Learning

Machine learning models get stale over time as spammers invent new tricks. We added a `/feedback` endpoint. If the model gets it wrong, the user can click "This is actually Spam!". We save this to a JSON file. The `/retrain` endpoint takes the original dataset, adds the new user feedback, and retrains the model from scratch!